

The Beginner's Guide to Fast Docker Image Layers

Docker image layers can make your builds lightning fast or painfully slow depending on how you arrange your Dockerfile. Understanding how layers work helps you ship code faster and save storage space.

What you'll learn:

- 1 Optimize Layer Caching by Ordering Steps
- 2 Keep Layers Lean by Combining Commands
- 3 Exclude Heavy Files with .dockerignore
- 4 Inspect Bloated Layers with Docker History

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Optimize Layer Caching by Ordering Steps

Docker executes Dockerfile commands top to bottom and caches each step. If line 4 changes, Docker re-runs line 4 and every line after it. Place rarely changed commands like package installs near the top, and frequently changed files like code at the bottom.

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
```

Cloud & DevOps Daily

Docker Image Layers: Why Your Builds Are Slow (and How to Fix)

Tuesday, September 22, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Keep Layers Lean by Combining Commands

Every RUN instruction creates a brand new image layer. If you download installer packages in one RUN command and delete them in the next, those files still exist hidden in the previous layer. Chain commands together so temp files are erased before the layer saves.

```
$RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
```

Cloud & DevOps Daily

Docker Image Layers: Why Your Builds Are Slow (and How to Fix)

Tuesday, September 22, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Exclude Heavy Files with .dockerignore

When you start a build, Docker copies your project files into the build engine. If you leave heavy folders like `node_modules` or `.git` in your directory, this transfer causes major delays. Adding a `.dockerignore` file tells Docker to bypass these unnecessary files entirely.

```
$# Save as .dockerignore in project root
node_modules
.git
.env
dist
```

Cloud & DevOps Daily

Docker Image Layers: Why Your Builds Are Slow (and How to Fix)

Tuesday, September 22, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Inspect Bloated Layers with Docker History

If your container image unexpectedly becomes huge, you can inspect each individual layer to locate the problem. The history command shows each line executed, its corresponding layer ID, and exact size added to the image.

```
$docker history my-app:latest
```


Cloud & DevOps Daily

Docker Image Layers: Why Your Builds Are Slow (and How to Fix)

Tuesday, September 22, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always copy package definition files before copying application source code.
- + Clean up installer cache files in the exact same RUN instruction where they were created.
- + Choose minimal base images like alpine or slim to reduce initial layer sizes.
- + Maintain a .dockerignore file in every repository to exclude local build artifacts.
- + Use multi-stage builds to discard build tools and export only runtime files.

Watch Out For

- ! Copying your whole code directory before running npm install or pip install.
- ! Trying to delete temporary files in a second RUN command instead of the same line.
- ! Forgetting to add node_modules or build folders to your .dockerignore file.
- ! Using full desktop or developer Linux distributions as base images.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh